Guide: Setting Up pyperformance in a Virtual Environment

Step 0 Copy the Image File:

Copy the prepared Ubuntu image from the server to your local machine:

```
scp <your_username>@naranja10.cslcs.technion.ac.il:/scratch/ece882-001/jammy-server-cloudimg-amd64-disk-kvm.img .
```

Step 1 Launch QEMU VM:

Start your QEMU environment using the copied image.
(see previous guide)

Step 2: Run perf with Python Debug Symbols:

To profile a specific benchmark, for instance json_dumps using perf, make sure you're using the debug build of Python (python3-dbg):

```
perf record -F 999 -g -- python3-dbg -m pyperformance run --bench json_dumps
```

After the run, export the report to a text file:

```
perf report --stdio > perf_report.txt
```

What you will see:

Thanks to the presence of Python debug symbols, the report will reveal internal Python function calls and stack traces used during the benchmark. This helps identify performance bottlenecks within Python itself.

HWSW Project: Benchmark Optimization, Analysis, and Hardware Acceleration Proposal

Overview:

In this project, you will explore how to analyze, optimize, and accelerate real Python code using two performance benchmarks from the **pyperformance** framework. These benchmarks simulate real-world Python workloads and test the performance of specific operations such as encryption, data serialization, logging, or memory usage. Your task is to deeply understand how each selected benchmark works, including the libraries, data structures, and algorithms it uses. This foundational understanding will help you identify which parts of the code are slow and why they may be inefficient.

Once you understand the behavior of the benchmarks, you'll use the performance profiling tool **perf** and flame graphs to visualize and analyze the program's runtime behavior. These tools will help you detect bottlenecks and specific parts of the code that slow down execution or use too many resources. After detecting these hotspots, your next goal is to optimize the benchmark. This could mean replacing an algorithm with a faster one, using a more efficient library, or restructuring the code for better performance.

To demonstrate the effect of your changes, you'll compare performance results from before and after the optimization. You will then take your analysis one step further by proposing a hardware acceleration solution for a selected part of the benchmark that could benefit from it. For example, you might suggest using a hardware accelerator to speed up encryption or memory access. You'll be expected to describe the hardware module's function, how it integrates with software, and its performance trade-offs. A block diagram is required to visualize how the hardware connects with the software system.

Lastly, all of your work will be documented and uploaded to a **Git** repository, including reports, scripts, optimized code, and a clear README file explaining how to reproduce your results. You are encouraged to use **AI** tools such as ChatGPT or GitHub Copilot to assist you during the project, but you must also submit the prompts or instructions you used when interacting with these tools. The goal is not just to write faster code, but to demonstrate a deep and thoughtful understanding of performance tuning and how software optimizations and hardware acceleration can work together to improve computational efficiency. In addition to the technical deliverables, you will be required to **prepare a short presentation** summarizing your work. During the presentation, you should be ready to **explain your decisions, discuss your analysis, and answer questions** about your optimizations and proposed hardware solutions.

Instructions:

1. **<u>Benchmark Analysis:</u>**

- Begin by reviewing and understanding the code for the selected benchmarks. You need to:
    - Identify the purpose of the benchmark and the libraries used.
    - Explore the data structures and algorithms employed in the benchmark.
    - If necessary, dive into any third-party libraries or dependencies used by the benchmark to fully understand their inner workings (e.g., libraries like numpy, pycrypto).
- Gain a deep understanding of the benchmark's behavior, which will be essential for detecting bottlenecks and proposing effective improvements.

2. **Understand pyperformance:**

- Familiarize yourself with how pyperformance works. You should learn how to:
    - Run the benchmark tests.
    - Capture performance results using pyperformance.
    - Interpret the performance data effectively
- Ensure that you are comfortable running benchmarks and comparing performance data, as this will form the foundation of your analysis and optimization process.
-

3. **Generate a Flame Graph:**

- Utilize pyperformance to generate flame graphs. These graphs will help visualize the performance hotspots in the benchmark by showing which parts of the code are consuming the most resources.
- Create flame graphs for each of the selected benchmarks to identify the most resource-intensive areas in the code.
-

4. **Detect Bottlenecks:**

- Analyze the generated flame graphs and other profiling data to detect performance bottlenecks.
- Pinpoint key areas in the benchmark where performance can be improved. These might include inefficient algorithms, slow function calls, or excessive memory usage.

5. Suggest Improvements:

- Based on your analysis, propose improvements to the benchmark code. You are encouraged to:
    - Use more efficient libraries, algorithms, or data structures.
    - Implement code refactoring or other optimization strategies.

- Provide clear and well-reasoned suggestions for optimizing the benchmark, backed by data that shows improvements such as reduced execution time or better memory efficiency.

6. Show Performance Improvements:

- After implementing the suggested optimizations, run the benchmarks again and capture the performance data.
- Demonstrate the improvements in performance after applying the optimizations, with concrete data comparing the optimized benchmark to the original one.
- If you can identify two or more benchmarks where your optimizations result in a performance improvement of **at least 7%**, that will be considered sufficient.

7. Propose Hardware Acceleration:

- For one or two key components of the benchmark, propose a hardware acceleration solution. Examples include:
    - Accelerating dictionary operations with specialized hardware.
    - Speeding up decompression using custom hardware modules.
    - Extend ISA with instructions can accelerate workloads but are not too workload-specific (e.g., multiple-accumulate)
    - Any other hardware accelerator that targets a significant performance bottleneck.
- Your proposal must include:
    - **Hardware description:** Implement the proposed hardware accelerator in **Verilog, SystemVerilog** or **PyXHDL**. Other hardware description languages or frameworks may be used **only with prior instructor approval**. The implementation does **not** need to be production-ready or suitable for tape-out, but it should represent a complete and logically consistent hardware design.
    - **Inputs and outputs:** Clearly define the inputs and outputs of the hardware accelerator, including data widths, interfaces, and expected operating frequency.
    - **Hardware architecture:** Describe the internal logic and operation of the accelerator. The implementation should include the main datapath and control logic necessary to demonstrate how the accelerator performs its task.
    - **Hardware/software interface:** Explain how the hardware module interacts with the existing software. Describe any required software modifications, APIs, drivers, memory-mapped interfaces, DMA transfers, or communication protocols needed to integrate the accelerator.

    - **Acceleration justification:** Explain why this component is a good candidate for hardware acceleration. Estimate the expected performance improvement and discuss any assumptions used in your analysis.
    - **Block diagram:** Include a block diagram illustrating the hardware accelerator, its interfaces, and its integration into the overall system.
    - **Performance/area/power trade-offs:** Discuss the expected trade-offs between performance, hardware complexity (area), operating frequency, and power consumption.

- **Notes:**
    - The goal of this assignment is to demonstrate a solid understanding of both the software and hardware aspects of hardware acceleration.
    - You are **not** expected to synthesize, fabricate, or physically test the hardware.
    - The hardware design should be sufficiently complete to define its functionality, interfaces, operating frequency, and internal logic, accompanied by an explanation in the report.

8. Git Repository and Documentation:

- Upload all your project files to a Git repository. Ensure that your repository contains:
    - The benchmark reports (report_<name_of_benchmark>.txt).
    - Shell scripts for running the benchmarks (script_<name_of_benchmark>.sh).
    - Any additional files you have created (Python scripts, configuration files, etc.).
    - A README.md file that explains the structure of the repository and provides instructions on how to run the scripts.
- Maintain good version control practices by using clear commit messages that demonstrate the development process and your understanding of the material. Properly structured commits and a well-organized repository will earn you bonus points (+5)

9. Presentation:

- You are required to prepare a 20–25 minutes presentation and present it at a scheduled time determined by the course staff.
- The best structure for your presentation is to simply follow the flow of your project work, from initial analysis to optimization and hardware proposal.
- During the session, you will be asked questions for 5–10 minutes, so be ready to explain your choices clearly.

- Make sure you have working code available to demonstrate and support your explanations (**No need to add all your code to the presentation!**). Your goal is to walk the course staff through your project as if you're explaining it to a fellow ECE student (someone with a technical background who didn't do the project themselves). Focus on teaching, guiding, and showing your understanding of the work.

10. AI Tools:

- You may use AI tools (such as ChatGPT, GitHub Copilot, etc.) to assist you in the project. However, you must provide the prompts or instructions you used to interact with the AI tool.
- The goal is to assess how well you understand the material, so the AI tools should be used as an aid, not a substitute for your own analysis and work.

Approved Benchmarks:

We have selected the following 10 benchmarks for you to choose from. You need to select **2 benchmarks** for this project. You can refer to the following link for more details about the benchmarks:

pyperformance Benchmark Documentation

Note: the ordering in this table carries no meaning

Index	Name of Benchmark
1	Raytrace
2	Deepcopy
3	Mdp
4	Pathlib
5	Pickle and pickle_dict
6	Pyflate
7	unpack_sequence
8	tornado_http
9	sqlite_synth
10	Nbody
11	Btree
12	deepblue
13	go

What You Need to Submit:

For each benchmark you select, you are required to submit the following:

1. Benchmark Reports:

- **File Name**: report_<name_of_benchmark>.txt
    - **Contents**:
        - Overview: Description of the benchmark, libraries used, and data structures employed.
        - Initial Analysis: Performance analysis, flame graphs, profiling data.
        - Optimizations: Description of improvements made, including any external libraries or algorithms used.
        - Performance Comparison: Show how performance improved after optimizations.
        - Hardware Acceleration Proposal: If applicable, explain the proposed hardware solution with inputs, outputs, trade-offs, and a block diagram.

        - Conclusion: Summarize the impact of the optimizations and hardware acceleration.

2. Benchmark Execution Scripts:

- **File Name**: script_<name_of_benchmark>.sh
    - **Contents**:
        - Environment setup and dependency installation.
        - Benchmark execution using pyperformance or other profiling tools.
        - Flame graph and performance data generation.
        - Post-optimization benchmark execution with performance comparison.

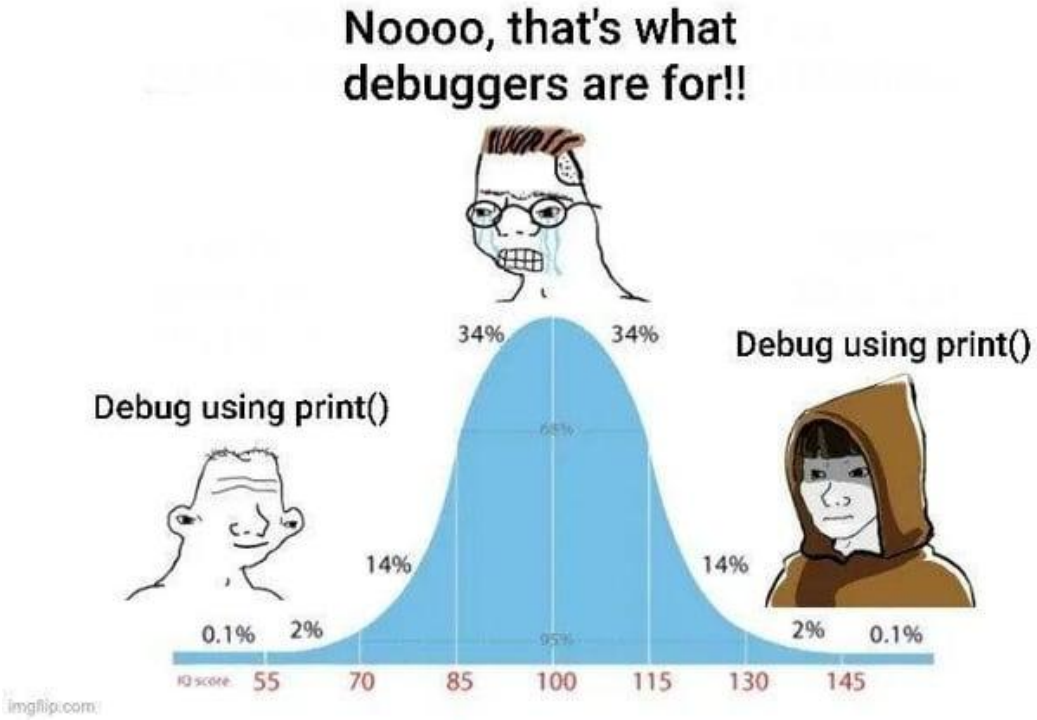
3. HW Files and Additional Files (Optional but encouraged):

- Python scripts (*.py), performance logs, configuration files, or any additional files that assist in your analysis or optimizations.

4. AI Tool Prompts:

- **File Name**: prompt.txt or prompt.docx
    - **Contents**:
        - Document the prompts or instructions used when interacting with AI tools.

The course staff will be available to assist you on the course forum.


Noooo, that's what debuggers are for!!
Debug using print()
Debug using print()
34%
34%
14%
14%
0.1%
2%
2%
0.1%
IQ score
55
70
85
100
115
130
145
imgflip.com